

Chapter 02 — M2: Entry Point Detection

Feature type: Mandatory analysis module (M2). **Source:** `backend/src/ai/nodes/m2EntryPoint.node.ts`, `backend/src/ai/prompts/m2Prompt.ts`, plus `resolveEntryContents()` in `backend/src/services/ai.service.ts` **Output type:** `IEntryPoint = { file, executionFlow[], description }`

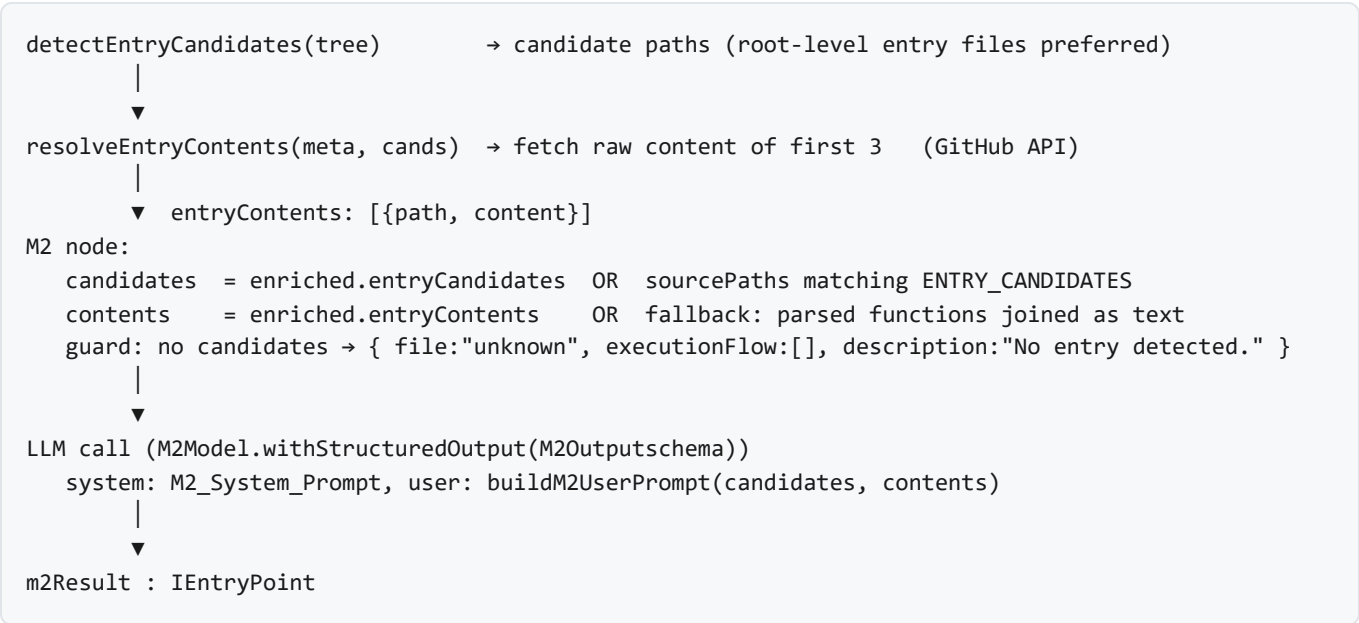
1. What it does

M2 answers "*where does this app actually start, and what happens on startup?*" It returns:

- `file` — the detected entry file (e.g. `server.ts`)
- `executionFlow` — an **ordered list** of startup steps in plain English
- `description` — a one-paragraph summary of the bootstrap logic

2. The key idea: content enrichment

M2 is the only module that reads **actual file contents**, not just paths. Before the graph runs, `ai.service.ts` → `resolveEntryContents()` fetches the raw source of the **top 3 entry candidates** from GitHub and passes them in as `entryContents: [{ path, content }]`. This lets the model read the real bootstrap code instead of guessing from a filename.



3. Entry-candidate detection

Two layers of candidates:

- **GitHub layer** (`github.service.detectEntryCandidates`): prefers **root-level** files matching `server/index/main/app.{js,ts}`, `main.py`, `app.py`, `Main.java`; falls back to any-depth matches if none at root.
- **Node layer** (`ENTRY_CANDIDATES` in the M2 node): the same filename list, used to filter `sourcePaths` when the enriched candidates are absent.

Fallback content: if no real file content is available, the node synthesizes pseudo-content by joining each candidate file's parsed function names — weaker, but keeps M2 producing something.

4. The model behind it

- **Model:** `M2Model` = NVIDIA `nemotron-3-super-120b-a12b`, `temperature: 0`.
 - **Why the big model:** M2 requires multi-step *reasoning* over real code (tracing an execution flow), so it uses the larger 120B Nemotron rather than the nano model M1 uses.
-

5. Failure behavior

- No candidates → `{ file:"unknown", executionFlow:[], description:"No entry detected." }` + error.
 - Thrown error → `{ file:"unknown", executionFlow:[], description:"" }` + `[M2] <message>`.
 - Missing at Combine → `m2` becomes `{ file:"", executionFlow:[], description:"" }` + warning.
-

6. Where it surfaces in the UI

`data.m2.file` + `data.m2.executionFlow[]` → the **entry-point flow** / **"Request Lifecycle"** panel on the Dashboard (rendered as an ordered step list).

7. Known gaps / improvement ideas

- Only the first 3 candidates get real content — a monorepo with many entry points loses detail.
- No language-specific bootstrap heuristics (e.g. detecting a framework's `createApp`).
- `executionFlow` ordering is trusted from the LLM; no validation that steps are causally ordered.